

Question 1: What is data cleaning and why is it performed before analysis? Explain how missing values are identified in Pandas using `isna().sum()` and `isna().mean()`, and describe the situations in which a column should be dropped instead of being imputed.

Answer :- 1. Data Cleaning & Its Importance

Data cleaning is the process of identifying, correcting, or removing corrupt, inaccurate, incomplete, or duplicate data from a dataset.

It is a critical step before performing any analysis or building machine learning models for several reasons:

- **Ensures Data Accuracy & Reliability:** "Garbage in, garbage out"—if models or analytical models run on dirty data, the conclusions drawn will be incorrect or misleading.
- **Prevents Algorithm Errors:** Many statistical methods and machine learning algorithms (e.g., Scikit-Learn estimators) cannot handle missing values or incorrect data types natively and will throw runtime errors.
- **Improves Model Performance:** Eliminating noise, outliers, and duplicates improves the generalization power and accuracy of predictive models.

2. Identifying Missing Values in Pandas

Pandas provides `isna()` (or its alias `isnull()`) to detect missing or **NaN** values across a `DataFrame`. Combining `isna()` with summary functions yields useful insights:

- **`df.isna().sum()`:**
 - **What it does:** Calculates the absolute count of missing values in each column.

- How it works: **isna()** returns a Boolean DataFrame where **True** represents missing values (\$1\$) and **False** represents non-missing values (\$0\$). **.sum()** adds up all the **True** values column-wise.
- Example Output:
- Python

```
df.isna().sum()
```

Output:

```
# age      5
```

```
# income   12
```

```
# gender    0
```

-
-

- **df.isna().mean():**

- What it does: Calculates the proportion/percentage of missing values in each column (values range between \$0.0\$ and \$1.0\$).
- How it works: It divides the sum of **True** values by the total number of rows in that column. Multiplying by \$100\$ gives the exact percentage of missing data.
- Example Output:
- Python

```
df.isna().mean() * 100
```

Output:

```
# age      5.0%
```

income 12.0%

gender 0.0%

-
-

3. When to Drop a Column Instead of Imputing

Imputation (filling in missing values with mean, median, mode, or predictive values) is generally preferred to preserve data volume. However, dropping a column entirely is better under the following circumstances:

1. Extremely High Missingness Ratio:

When a column is missing a vast majority of its data (e.g., > 60% to 70% missing). Imputing such a large portion introduces heavy bias and artificial patterns into the data.

2. Irrelevant or Non-Informative Feature:

If a column with missing values has little to no predictive power or business relevance to the target variable (e.g., a secondary optional feedback field in a customer demographic dataset).

3. Data Cannot Be Logically Estimated:

When values are missing completely at random or represent unique identifiers/un-imputable characteristics (e.g., missing social security numbers or unique transaction IDs) where mean/mode substitution makes no statistical sense.

4. Data Leakage or High Redundancy:

If the information provided by the missing column is already captured accurately by another complete feature in the dataset.

Question 2: Explain median imputation and mode imputation with suitable examples. Why is the median preferred for numerical columns such as age, and why is the mode used for categorical columns such as embarked?

Answer :- 1. Median Imputation

Median imputation is the process of replacing missing values (NaN) in a numerical column with the median (middle value) of the available non-missing data.

- **Example:**

Consider a dataset of ages: [20, 22, 25, 28, NaN, 85].

- First, calculate the median of the known values [20, 22, 25, 28, 85]:
$$\text{Median} = 25$$
- Replace NaN with 25:
$$\text{Result} = [20, 22, 25, 28, \mathbf{25}, 85]$$

2. Mode Imputation

Mode imputation is the process of replacing missing values in a categorical or discrete column with the mode (the most frequently occurring category).

- **Example:**

Consider a dataset of embarkation ports: ['Southampton', 'Cherbourg', 'Southampton', NaN, 'Southampton', 'Queenstown'].

- Determine the most common category:
$$\text{Mode} = \text{'Southampton'}$$

- Replace NaN with 'Southampton':

$$\text{Result} = [\text{'Southampton'}, \text{'Cherbourg'}, \text{'Southampton'}, \textbf{\text{'Southampton'}}, \text{'Southampton'}, \text{'Queenstown'}]$$

3. Why Median is Preferred for Numerical Columns (e.g., age)

- **Robustness to Outliers & Skewness:** Real-world continuous data like age or income often has skewed distributions or extreme outliers (e.g., a few elderly passengers aged 80–90 or wealthy individuals). The mean is heavily pulled towards outliers, which distorts the central tendency. The median represents the exact midpoint and is robust against extreme values.
- **Preserves Data Centrality:** Imputing with the median introduces less bias than the mean when data is non-normally distributed, preserving the central tendency of the dataset.

4. Why Mode is Used for Categorical Columns (e.g., embarked)

- **Non-Numeric Nature:** Categorical variables contain discrete text strings, labels, or classes (e.g., 'Southampton', 'Cherbourg'). Mathematical operations like sum, mean, or median cannot be computed on string labels.
- **Maximum Likelihood Estimate:** For discrete classes, substituting the most common category (mode) minimizes the overall error rate assuming the data is missing at random, making it the most statistically probable value to assign to an unknown record.

Question 3: What is the difference between `map()` and `apply()`? Explain with examples where each one should be used, and describe what happens when a value is missing from the dictionary passed to `map()`.

Answer :- . Key Differences: `map()` vs. `apply()`

Feature	<code>map()</code>	<code>apply()</code>
Supported Objects	Only Pandas Series .	Pandas Series and DataFrame .
Accepted Inputs	Functions, Dictionaries, or Series .	Custom or built-in Python/NumPy Functions.
Primary Scope	Element-wise substitutions and value mappings.	Complex axis-wise computations or custom functions.
Performance	Optimized and faster for element-wise dictionary mapping.	Extremely versatile, but can be slower on large DataFrames.

2. Examples & When to Use Each

When to use `map()`

Use `map()` for simple element-wise value substitution or lookup using a dictionary or mapping function on a single Series.

- Example: Converting categorical codes or labels to descriptive strings using a dictionary.

Python

```
import pandas as pd
```

```
s = pd.Series(['M', 'F', 'M', 'F'])
```

```
gender_map = {'M': 'Male', 'F': 'Female'}
```

```
# Using map() with a dictionary
```

```
s_mapped = s.map(gender_map)
```

```
print(s_mapped)
```

```
# Output:
```

```
# 0    Male
```

```
# 1    Female
```

```
# 2    Male
```

```
# 3    Female
```

When to use `apply()`

Use `apply()` when applying a custom function across rows/columns of a `DataFrame`, or when performing element-wise transformations on a `Series` using complex conditional logic or external arguments.

- Example 1 (On a `Series`): Categorizing numerical values using custom conditional logic.

Python

```
def classify_age(age):
```

```
    return 'Adult' if age >= 18 else 'Minor'
```

```
df = pd.DataFrame({'age': [12, 25, 17, 30]})
```

```
df['age_group'] = df['age'].apply(classify_age)
```

- Example 2 (On a `DataFrame`): Row-wise calculation combining multiple columns.

Python

```
# Compute total cost across rows (axis=1)
```

```
df_sales = pd.DataFrame({'price': [100, 200], 'qty': [2, 3]})
```

```
df_sales['total'] = df_sales.apply(lambda row: row['price'] *  
row['qty'], axis=1)
```

3. What Happens When a Value is Missing from a Mapping Dictionary?

When a dictionary is passed to `map()`, any value in the Series that is NOT present as a key in the dictionary will be replaced with `NaN` (or `None` for non-numeric dtypes).

Unlike Python standard dictionaries, `map()` does not raise a `KeyError`; it silently turns unmapped elements into missing values.

- Example:

Python

```
s = pd.Series(['apple', 'banana', 'cherry'])
```

```
fruit_prices = {'apple': 1.5, 'banana': 0.75} # 'cherry' is missing
```

```
result = s.map(fruit_prices)
```

```
print(result)
```

Output:

Plaintext

```
0    1.50
```

```
1    0.75
```

```
2    NaN
```

dtype: float64

Tip: If you want to preserve the original values for unmapped keys instead of converting them to NaN, use `replace()` instead of `map()`:

Python

```
s.replace(fruit_prices)
```

Question 4: Explain binning in Pandas. Describe the difference between `pd.cut()` and `pd.qcut()` with suitable examples, and mention one real-world situation in which each one is useful.

Answer :- Binning (or discretization) is the process of grouping continuous numerical values into discrete intervals ("bins" or categories). It converts continuous numerical data into categorical features, which helps reduce noise, handle non-linear relationships, and simplify analytical modeling or reporting.

2. Difference Between `pd.cut()` and `pd.qcut()`

Feature	<code>pd.cut()</code> (Equal-Width Binning)	<code>pd.qcut()</code> (Equal-Frequency Binning)
---------	--	--

Binning Logic	Divides data into bins of equal width or based on pre-defined fixed intervals .	Divides data into bins based on sample quantiles/percentiles .
Data Distribution	Each bin covers the same range of values , but bins can contain a different number of data points.	Each bin contains approximately the same number of records , but the range/width of each bin varies.
Outlier Handling	Sensitive to extreme outliers, which can stretch the range and leave intermediate bins sparse or empty.	Robust against outliers, as extreme values naturally fall into the top or bottom quantile bins without distorting bin counts.

3. Examples & Real-World Use Cases

1. `pd.cut()` — Equal-Width Binning

Divides the value range into custom intervals or fixed equal-width spans.

- **Code Example:**
- Python

```
import pandas as pd
```

```
ages = pd.Series([5, 15, 22, 35, 45, 62, 70])
```

```
# Custom fixed boundaries: 0–18, 18–60, 60–100
```

```
bins = [0, 18, 60, 100]
```

```
labels = ['Minor', 'Adult', 'Senior']
```

```
df_cut = pd.cut(ages, bins=bins, labels=labels)
```

```
print(df_cut)
```

-
-
- **Output:**
- Plaintext

```
0    Minor
```

- 1 Minor
- 2 Adult
- 3 Adult
- 4 Adult
- 5 Senior
- 6 Senior

dtype: category

-
-
- **Real-World Situation: Demographic Age Grouping:**
Classifying individuals into predefined legal/regulatory age categories (e.g., 'Minor', 'Adult', 'Senior') based on explicit age thresholds, regardless of how many people fall into each age group.

2. `pd.qcut()` — Equal-Frequency Binning

Divides data into quantiles so that each bin contains an equal share of the total records (e.g., quartiles or deciles).

- **Code Example:**
- Python

```
import pandas as pd
```

```
scores = pd.Series([10, 20, 35, 40, 50, 80, 90, 95])
```

```
# Divide into 4 bins with equal number of records (Quartiles)
```

```
df_qcut = pd.qcut(scores, q=4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
```

```
print(df_qcut)
```

-
-
- **Output:**
- Plaintext

```
0    Q1
```

```
1    Q1
```

```
2    Q2
```

```
3    Q2
```

```
4    Q3
```

```
5    Q3
```

```
6    Q4
```

```
7    Q4
```

```
dtype: category
```

-
-

- **Real-World Situation: Customer Income Segmentation (RFM Analysis):** Segmenting customers into equal economic tiers (e.g., 'Low Income', 'Middle Income', 'High Income') where you require exactly 33.3% of the customer base in each group to run balanced marketing campaigns or tier-based reward programs.

Question 5: Explain the difference between `agg()`, `transform()` and `filter()` in a `groupby` operation. Support your answer with one example of each.

Answer :- 1. Differences Between `agg()`, `transform()`, and `filter()`

When performing `GroupBy` operations in Pandas, `agg()`, `transform()`, and `filter()` serve fundamentally different purposes regarding output shape and operation type:

Method	Primary Purpose	Output Shape / Dimensions	Common Use Cases
<code>agg()</code>	Aggregation: Reduces group data down to single summary statistic(s).	Returns one row per group.	Calculating group totals, averages, or custom summary stats across multiple columns.

transform()	Broadcast Transformation: Applies a function to each group and broadcasts the calculated value(s) back to match the original DataFrame shape.	Returns same number of rows as the original DataFrame.	Feature engineering like group-level standardization (z-score), calculating group means, or percentage contribution.
filter()	Group Filtering: Filters out entire groups based on a boolean condition evaluated on group properties.	Returns a subset of original rows (entire groups included or excluded).	Filtering out low-volume categories, removing groups with low sample size, or keeping groups where total sum exceeds a threshold.

2. Examples for Each Method

Let's use a sample DataFrame:

Python

```
import pandas as pd
```

```
df = pd.DataFrame({  
    'category': ['A', 'A', 'A', 'B', 'B', 'C'],  
    'sales': [100, 200, 300, 15, 25, 500]  
})
```

1. `agg()` Example

Calculates summary statistics (e.g., total sales and mean sales) per group.

Python

```
# Returns a aggregated DataFrame with 1 row per category
```

```
df_agg = df.groupby('category').agg({'sales': ['sum', 'mean']})
```

```
print(df_agg)
```

Output:

Plaintext

	sales	
	sum	mean
category		
A	600	200.0
B	40	20.0
C	500	500.0

2. `transform()` Example

Calculates the group mean sales and assigns it back to each row of the original DataFrame.

Python

```
# Returns a Series with 6 rows matching the original DataFrame index
```

```
df['category_avg_sales'] =  
df.groupby('category')['sales'].transform('mean')  
  
print(df)
```

Output:

Plaintext

	category	sales	category_avg_sales
0	A	100	200.0
1	A	200	200.0
2	A	300	200.0
3	B	15	20.0
4	B	25	20.0
5	C	500	500.0

3. `filter()` Example

Keeps only the groups whose total sales exceed 100 (filters out Category 'B').

Python

Returns all original rows belonging to groups meeting the condition

```
df_filtered = df.groupby('category').filter(lambda g:
g['sales'].sum() > 100)
```

```
print(df_filtered)
```

Output:

Plaintext

category sales

0 A 100

1 A 200

2 A 300

5 C 500

Question 6: Use the following dataset: import pandas as pd df = pd.DataFrame({ 'region':

['North','North','South','South','East','East','East'], 'category':

['A','B','A','A','B','A','B'], 'sales': [100, 150, 200, 250, 120, 90, 300],

'discount': [5, 10, 0, 2, 3, 1, 4], 'quantity': [10, 12, 15, 20, 8, 9, 14]

}) Write a Python program to: 1. Group the data by region and

category and find the total sales of every group. 2. Find the

number of records present in every group. 3. Create a region-wise

summary containing total_sales, avg_discount and max_qty using

named aggregation. 4. Find the mean, min and max sales of

every region–category pair.

Answer :- Python

import pandas as pd

Initial Dataset

```
df = pd.DataFrame({  
    'region': ['North', 'North', 'South', 'South', 'East', 'East', 'East'],  
    'category': ['A', 'B', 'A', 'A', 'B', 'A', 'B'],  
    'sales': [100, 150, 200, 250, 120, 90, 300],  
    'discount': [5, 10, 0, 2, 3, 1, 4],  
    'quantity': [10, 12, 15, 20, 8, 9, 14]  
})
```

1. Group by region and category and find the total sales of every group

```
total_sales_per_group = df.groupby(['region',  
    'category'])['sales'].sum().reset_index()
```

```
print("--- 1. Total Sales per Group (region, category) ---")
```

```
print(total_sales_per_group)
```

```
print()
```

2. Find the number of records present in every group

```
record_counts = df.groupby(['region',  
    'category']).size().reset_index(name='record_count')
```

```
print("--- 2. Number of Records per Group ---")
```

```
print(record_counts)
```

```
print()
```

3. Create a region-wise summary containing total_sales, avg_discount, and max_qty using named aggregation

```
region_summary = df.groupby('region').agg(
```

```
    total_sales=('sales', 'sum'),
```

```
    avg_discount=('discount', 'mean'),
```

```
    max_qty=('quantity', 'max')
```

```
).reset_index()
```

```
print("--- 3. Region-wise Summary (Named Aggregation) ---")
```

```
print(region_summary)
```

```
print()
```

4. Find the mean, min, and max sales of every region-category pair

```
sales_stats_per_pair = df.groupby(['region',  
'category'])['sales'].agg(['mean', 'min', 'max']).reset_index()
```

```
print("--- 4. Mean, Min, and Max Sales per Region-Category Pair ---")
```

```
print(sales_stats_per_pair)
```

Expected Output:

Plaintext

--- 1. Total Sales per Group (region, category) ---

	region	category	sales
0	East	A	90
1	East	B	420
2	North	A	100
3	North	B	150
4	South	A	450

--- 2. Number of Records per Group ---

	region	category	record_count
0	East	A	1
1	East	B	2
2	North	A	1
3	North	B	1
4	South	A	2

--- 3. Region-wise Summary (Named Aggregation) ---

	region	total_sales	avg_discount	max_qty
0	East	510	2.666667	14
1	North	250	7.500000	12
2	South	450	1.000000	20

--- 4. Mean, Min, and Max Sales per Region-Category Pair ---

	region	category	mean	min	max
0	East	A	90.0	90	90
1	East	B	210.0	120	300
2	North	A	100.0	100	100
3	North	B	150.0	150	150
4	South	A	225.0	200	250

Question 7: Using the same dataset given in Question 6, write a Python program to: 1. Add a column `region_avg_sales` containing the average sales of the region of each row. 2. Add a column `pct_contribution` showing the share of each row in its region's total sales. 3. Add a column `rank_region_sales` that ranks the sales within each region in descending order. 4. Keep only those regions that contain more than two records

Answer:- Python

```
import pandas as pd
```

```
# Dataset from Question 6
```

```
df = pd.DataFrame({  
    'region': ['North', 'North', 'South', 'South', 'East', 'East', 'East'],  
    'category': ['A', 'B', 'A', 'A', 'B', 'A', 'B'],  
    'sales': [100, 150, 200, 250, 120, 90, 300],  
    'discount': [5, 10, 0, 2, 3, 1, 4],  
    'quantity': [10, 12, 15, 20, 8, 9, 14]  
})
```

```
# 1. Add a column region_avg_sales containing the average sales of  
the region of each row
```

```
df['region_avg_sales'] = df.groupby('region')['sales'].transform('mean')
```

2. Add a column pct_contribution showing the share of each row in its region's total sales

```
df['pct_contribution'] = (df['sales'] /  
df.groupby('region')['sales'].transform('sum')) * 100
```

3. Add a column rank_region_sales that ranks the sales within each region in descending order

```
df['rank_region_sales'] =  
df.groupby('region')['sales'].rank(ascending=False,  
method='min').astype(int)
```

4. Keep only those regions that contain more than two records

```
df_filtered = df.groupby('region').filter(lambda g: len(g) > 2)
```

```
print(df_filtered)
```

Output Explanation & Detailed Results:

1. **region_avg_sales**: Uses `groupby().transform('mean')` to broadcast the mean sales of each region (**East**: \$170.0\$, **North**: \$125.0\$, **South**: \$225.0\$) across all individual rows.

2. **pct_contribution**: Divides the individual row sales by the region's total sales using `groupby().transform('sum')` and multiplies by \$100\$. (e.g., for **East**, total sales = \$510\$, so $\$300 / 510 \times 100 \approx 58.82\%$).
3. **rank_region_sales**: Ranks rows within each region in descending order using `.rank(ascending=False)` (\$1\$ being the highest sales in that region).
4. **df_filtered**: Uses `.filter(lambda g: len(g) > 2)` to retain only groups with more than 2 records. Since **East** has 3 records while **North** and **South** have 2 records each, only the rows corresponding to **East** are kept.

Plaintext

	region	category	sales	discount	quantity	region_avg_sales	pct_contribution	rank_region_sales
4	East	B	120	3	8	170.0	23.529412	2
5	East	A	90	1	9	170.0	17.647059	3
6	East	B	300	4	14	170.0		

Question 8: Use the following dataset: `df = pd.DataFrame({'region': ['North','North','South','South','East','East'], 'month': ['Jan','Feb','Jan','Feb','Jan','Feb'], 'sales': [100, 120, 200, 220, 150, 180], 'discount': [5, 7, 3, 4, 6, 8] })` Write a Python program to: 1. Convert the data into a wide format using `pivot()`, with region as the index and the months as columns. 2. Create a pivot table of total sales along with an overall Total row and column. 3. Create a pivot table of the average discount in which the missing combinations are filled with 0. 4. Explain in which situation `pivot()` fails but `pivot_table()` works.

Answer :- Python

```
import pandas as pd
```

```
# Initial Dataset
```

```
df = pd.DataFrame({  
    'region': ['North', 'North', 'South', 'South', 'East', 'East'],  
    'month': ['Jan', 'Feb', 'Jan', 'Feb', 'Jan', 'Feb'],  
    'sales': [100, 120, 200, 220, 150, 180],  
    'discount': [5, 7, 3, 4, 6, 8]  
})
```

```
# 1. Convert data into wide format using pivot(), region as index  
and month as columns
```

```
pivot_df = df.pivot(index='region', columns='month',  
values='sales')
```

```
print("--- 1. Wide Format using pivot() ---")
```

```
print(pivot_df)
```

```
print()
```

2. Pivot table of total sales along with an overall Total row and column (margins=True)

```
pivot_sales_totals = df.pivot_table(  
    index='region',  
    columns='month',  
    values='sales',  
    aggfunc='sum',  
    margins=True,  
    margins_name='Total'  
)  
  
print("--- 2. Total Sales with Overall Totals ---")  
  
print(pivot_sales_totals)  
  
print()
```

3. Pivot table of average discount with missing combinations filled with 0

```
pivot_discount_avg = df.pivot_table(  
    index='region',  
    columns='month',
```

```
    values='discount',  
    aggfunc='mean',  
    fill_value=0  
)  
  
print("--- 3. Average Discount (fill_value=0) ---")  
  
print(pivot_discount_avg)
```

Execution Results:

Plaintext

--- 1. Wide Format using pivot() ---

month	Feb	Jan
-------	-----	-----

region		
--------	--	--

East	180	150
------	-----	-----

North	120	100
-------	-----	-----

South	220	200
-------	-----	-----

--- 2. Total Sales with Overall Totals ---

month	Feb	Jan	Total
-------	-----	-----	-------

region			
--------	--	--	--

East	180	150	330
North	120	100	220
South	220	200	420
Total	520	450	970

--- 3. Average Discount (fill_value=0) ---

month	Feb	Jan
-------	-----	-----

region		
--------	--	--

East	8	6
------	---	---

North	7	5
-------	---	---

South	4	3
-------	---	---

4. Explanation: When `pivot()` Fails but `pivot_table()` Works

- `pivot()` Fails on Duplicate Entries:
`pivot()` requires every combination of index and column values to be strictly unique. If the dataset contains duplicate rows for the same (`index`, `column`) pair (e.g., two rows with 'North' and 'Jan'), `pivot()` raises a `ValueError: Index contains duplicate entries, cannot reshape` because it does not aggregate multiple values.

- **pivot_table()** Handles Duplicates:
pivot_table() automatically handles non-unique combinations by applying an aggregation function (**aggfunc**, such as **'mean'**, **'sum'**, or **'count'**) to aggregate duplicate records into a single summary value. Additionally, **pivot_table()** provides extra features like adding summary totals (**margins=True**) and filling missing combinations (**fill_value=0**).

Question 9: Explain rolling, expanding and cumulative operations in Pandas. Using the sales data of Question 8, write a Python program to create the columns **rolling_mean_3**, **expanding_avg**, **cum_sum** and **cum_max**, and explain why the first two values of the rolling mean are NaN and how **min_periods=1** changes the output.

Answer :- 1. Explanation of Windowing Operations in Pandas

- **Rolling Operations (rolling):**
Computes statistics over a moving/sliding window of a fixed size (e.g., a 3-period window). As you iterate down the series, the oldest data points drop off as new ones enter, keeping the total window size constant.
- **Expanding Operations (expanding):**
Computes statistics over a growing window that starts from the beginning of the series and includes all preceding data points up to the current row. The window size grows by 1 for each successive row.

- Cumulative Operations (**cumsum**, **cummax**, **cummin**, **cumprod**):
Specialized, highly optimized vector operations that return running aggregate metrics (e.g., cumulative sum, cumulative maximum) calculated from the start of the series up to the current index.

2. Python Program

Python

```
import pandas as pd
```

```
# Dataset from Question 8
```

```
df = pd.DataFrame({  
    'region': ['North', 'North', 'South', 'South', 'East', 'East'],  
    'month': ['Jan', 'Feb', 'Jan', 'Feb', 'Jan', 'Feb'],  
    'sales': [100, 120, 200, 220, 150, 180],  
    'discount': [5, 7, 3, 4, 6, 8]  
})
```

```
# 1. Rolling Mean (window size of 3)
```

```
df['rolling_mean_3'] = df['sales'].rolling(window=3).mean()
```

2. Expanding Average

```
df['expanding_avg'] = df['sales'].expanding().mean()
```

3. Cumulative Sum

```
df['cum_sum'] = df['sales'].cumsum()
```

4. Cumulative Maximum

```
df['cum_max'] = df['sales'].cummax()
```

```
print(df[['sales', 'rolling_mean_3', 'expanding_avg', 'cum_sum',  
'cum_max']])
```

3. Program Output

Plaintext

	sales	rolling_mean_3	expanding_avg	cum_sum	cum_max
0	100	NaN	100.000000	100	100
1	120	NaN	110.000000	220	120
2	200	140.000000	140.000000	420	200
3	220	180.000000	160.000000	640	220

4	150	190.000000	158.000000	790	220
5	180	183.333333	161.666667	970	220

4. Why the First Two Values are NaN & How min_periods=1 Changes the Output

- Why the first two values of `rolling_mean_3` are NaN:
By default, `rolling(window=3)` requires a full window of at least 3 non-null observations (`min_periods` defaults to window size).
 - At index **0** (1st row), only 1 observation is available (\$100\$).
 - At index **1** (2nd row), only 2 observations are available (\$100, 120\$).
Because neither row satisfies the required threshold of 3 observations, Pandas returns NaN. The calculation produces a valid numeric result starting only at index **2** ($(100 + 120 + 200) / 3 = 140.0$).
- How `min_periods=1` changes the output:
Setting `min_periods=1` allows Pandas to compute the aggregation as long as there is at least 1 valid observation in the current window.
 - Index **0**: Averages 1 value $\rightarrow 100.0 / 1 = 100.0$
 - Index **1**: Averages 2 values $\rightarrow (100 + 120) / 2 = 110.0$
 - Index **2** onwards: Averages 3 values normally.
- Python

Code with min_periods=1

```
df['rolling_mean_3_min1'] = df['sales'].rolling(window=3,  
min_periods=1).mean()
```

-
- Updated Output:
- Plaintext

	sales	rolling_mean_3_min1
--	-------	---------------------

0	100	100.000000
---	-----	------------

1	120	110.000000
---	-----	------------

2	200	140.000000
---	-----	------------

3	220	180.000000
---	-----	------------

4	150	190.000000
---	-----	------------

• 5	180	183.333333
-----	-----	------------

-

Question 10: Use the following datasets: import pandas as pd
import numpy as np sales = pd.DataFrame({ 'order_id': [1, 2, 3, 4, 5, 6], 'region': ['North','South','East','North','South','East'],
'category': ['A','B','A','B','A','B'], 'sales': [100, 250, np.nan, 160, 200, 300], 'quantity': [10, 20, 12, 10, 15, 14], 'rep_id': [101, 102, 103, 101, 102, 103] }) reps = pd.DataFrame({ 'rep_id': [101, 102, 103], 'rep_name': ['Aditi', 'Rahul', 'Sneha'], 'experience': [5, 2, 7] })
Write a single Python program to: 1. Display the number of missing values in every column and fill the missing sales with the median. 2. Merge sales with reps on rep_id using a left join. 3. Create a column revenue_per_unit as sales / quantity. 4. Create a region-wise summary containing total_sales, avg_sales and orders. 5. Create a pivot table of total sales with region as the index and category as the columns. 6. Print the name of the representative who generated the highest total sales.

Answer :- Python

```
import pandas as pd
```

```
import numpy as np
```

Initial Datasets

```
sales = pd.DataFrame({  
    'order_id': [1, 2, 3, 4, 5, 6],  
    'region': ['North', 'South', 'East', 'North', 'South', 'East'],  
    'category': ['A', 'B', 'A', 'B', 'A', 'B'],  
    'sales': [100, 250, np.nan, 160, 200, 300],  
    'quantity': [10, 20, 12, 10, 15, 14],  
    'rep_id': [101, 102, 103, 101, 102, 103]  
})
```

```
reps = pd.DataFrame({  
    'rep_id': [101, 102, 103],  
    'rep_name': ['Aditi', 'Rahul', 'Sneha'],  
    'experience': [5, 2, 7]  
})
```

```
})
```

1. Display missing values per column and fill missing sales with median

```
print("--- 1. Missing Values per Column ---")
```

```
print(sales.isna().sum())
```

```
print()
```

```
sales_median = sales['sales'].median()
```

```
sales['sales'] = sales['sales'].fillna(sales_median)
```

2. Merge sales with reps on rep_id using a left join

```
merged_df = pd.merge(sales, reps, on='rep_id', how='left')
```

3. Create column revenue_per_unit as sales / quantity

```
merged_df['revenue_per_unit'] = merged_df['sales'] /  
merged_df['quantity']
```

```
print("--- Data after Merging and Revenue Calculation ---")
```

```
print(merged_df[['order_id', 'rep_name', 'sales', 'quantity',  
'revenue_per_unit']])
```

```
print()
```

4. Create region-wise summary containing total_sales, avg_sales, and orders

```
region_summary = merged_df.groupby('region').agg(  
    total_sales=('sales', 'sum'),  
    avg_sales=('sales', 'mean'),  
    orders=('order_id', 'count')  
).reset_index()
```

```
print("--- 4. Region-wise Summary ---")
```

```
print(region_summary)
```

```
print()
```

5. Create a pivot table of total sales with region as index and category as columns

```
sales_pivot = merged_df.pivot_table(  
    index='region',  
    columns='category',  
    values='sales',  
    aggfunc='sum',  
    fill_value=0  
)
```

```
index='region',  
columns='category',  
values='sales',  
aggfunc='sum'  
)
```

```
print("--- 5. Pivot Table (Total Sales) ---")  
print(sales_pivot)  
print()
```

6. Print the name of the representative who generated the highest total sales

```
top_rep_series = merged_df.groupby('rep_name')['sales'].sum()  
top_rep_name = top_rep_series.idxmax()  
top_rep_sales = top_rep_series.max()
```

```
print("--- 6. Representative with Highest Total Sales ---")
```

```
print(f"Top Representative: {top_rep_name} with Total Sales of {top_rep_sales}")
```

Execution Output:

Plaintext

--- 1. Missing Values per Column ---

order_id 0

region 0

category 0

sales 1

quantity 0

rep_id 0

dtype: int64

--- Data after Merging and Revenue Calculation ---

	order_id	rep_name	sales	quantity	revenue_per_unit
0	1	Aditi	100.0	10	10.000000
1	2	Rahul	250.0	20	12.500000
2	3	Sneha	200.0	12	16.666667
3	4	Aditi	160.0	10	16.000000
4	5	Rahul	200.0	15	13.333333

5	6	Sneha	300.0	14	21.428571
---	---	-------	-------	----	-----------

--- 4. Region-wise Summary ---

	region	total_sales	avg_sales	orders
0	East	500.0	250.0	2
1	North	260.0	130.0	2
2	South	450.0	225.0	2

--- 5. Pivot Table (Total Sales) ---

category	A	B
region		
East	200.0	300.0
North	100.0	160.0
South	200.0	250.0

--- 6. Representative with Highest Total Sales ---

Top Representative: Sneha with Total Sales of 500.0